

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Christopher Becerra Catalan

11 de septiembre de 2026

22:16

1. Introducción

El informe detalla el análisis experimental de cuatro algoritmos de ordenamiento unidimensional y dos algoritmos de multiplicación de matrices cuadradas. El objetivo fundamental es ir más allá de la notación asintótica teórica para evaluar el rendimiento práctico de estas implementaciones sobre hardware de un computador o dispositivo. Al someter los algoritmos a diferentes distribuciones y volúmenes de datos, se busca relacionar directamente su complejidad matemática con el tiempo de ejecución del algoritmo (programa), demostrando que factores como las constantes, la gestión de memoria y los peores casos teóricos tienen un impacto crítico en el poder de cómputo. Para poder entender esto a continuación se presenta el entorno experimental con el que se trabajó.

2. Entorno experimental

Las pruebas de rendimiento se ejecutaron en un entorno Linux Ubuntu 24.04.4 LTS mediante el compilador estándar de C++, El hardware corresponde a un HP HP Pavilion Gaming Laptop 15-cx0xxx, procesador Intel® Core™ i5-8300H $\times$ 8 y 16 GB de RAM. Los casos de prueba para el problema de ordenamiento abarcaron arreglos desde $N=10$ hasta 10.000.000 de elementos, evaluando distribuciones ascendentes, descendentes y aleatorias. Para la multiplicación de matrices, se procesaron estructuras desde la dimensión $N = 2^1$ hasta $N = 2^8$ bajo formatos densos, diagonales y dispersos.

3. Resultados y Análisis

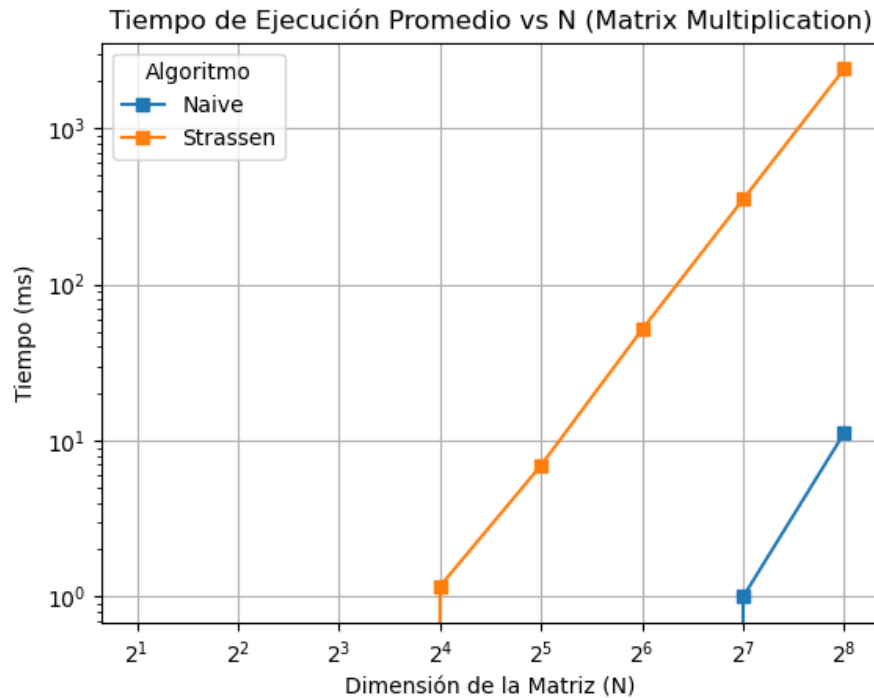


Figura 1: Tiempo de ejecución vs Dimensión de la matriz para los algoritmos Naive y Strassen.

Algoritmo	$N = 2$	$N = 4$	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$	$N = 256$
Naive	0.000	0.000	0.000	0.000	0.000	0.000	1.056	11.444
Strassen	0.000	0.000	0.000	1.000	7.333	53.111	344.444	2312.167

Cuadro 1: Tiempos de ejecución (en milisegundos) para multiplicación de matrices. Se observa que el método Naive mantiene tiempos inferiores, demostrando el sobre costo operativo de Strassen en dimensiones previas al umbral de transición.

3.1. Multiplicación de Matrices: El peso de las constantes y el umbral empírico

Como se detalla en la Tabla 1 y se ilustra en la Figura 1, el algoritmo Naive supera de forma aplastante a Strassen en la práctica para las dimensiones evaluadas. Para la dimensión de matriz máxima de $N=256$, el método Naive finaliza en 11.444 ms, mientras que Strassen se dispara a 2312.167 ms. Aunque teóricamente Strassen reduce la cantidad de multiplicaciones escalares de $O(n^3)$ a $O(n^{2.81})$, su sobrecarga estructural y la creación dinámica de submatrices saturaron la memoria y el procesador antes de que el volumen de datos lograra compensar su costo operativo inicial.

Esto demuestra mediante la experimentación (empíricamente) que, para dimensiones moderadas, las operaciones directas e iterativas superan a los enfoques de dividir y conquistar teóricos. La regla del

límite establece que la función con menor orden de crecimiento dominara, pero la constante oculta en la ecuación de recurrencia de Strassen $T(n) = 7T(n/2) + O(n^2)$ es tan alta que el *crossover point* (umbral de transición) donde Strassen se vuelve superior no se alcanza dentro del rango $N \leq 256$.

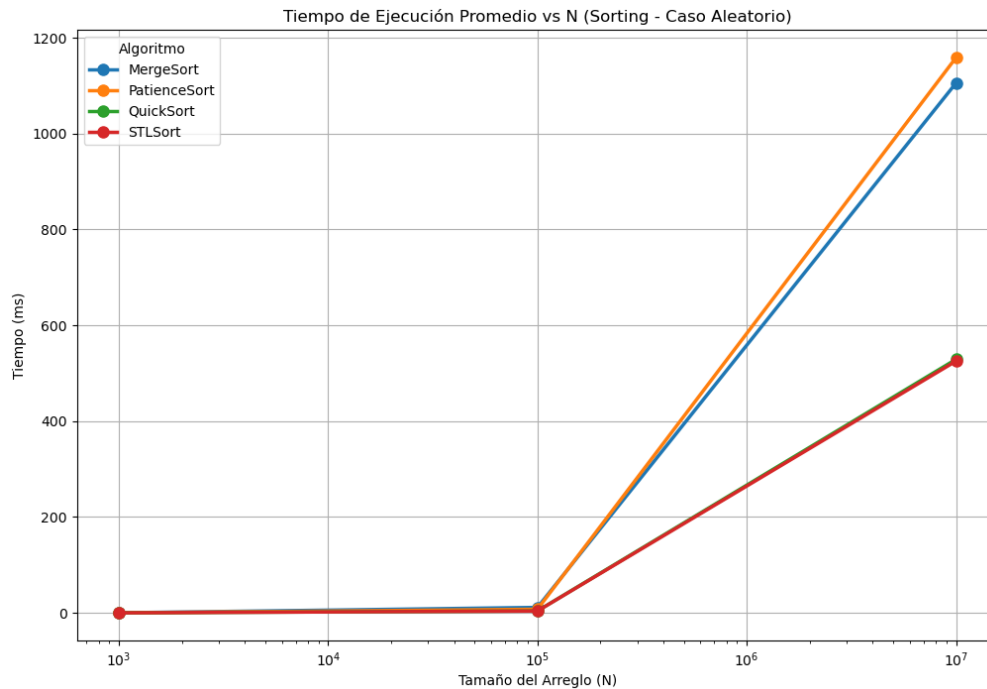


Figura 2: Tiempo de ejecución vs Tamaño del arreglo para algoritmos de ordenamiento.

Algoritmo	$N = 1000$	$N = 100000$	$N = 10000000$
MergeSort	0.000	0.011	1.095
PatienceSort	0.000	0.007	1.159
QuickSort	0.000	0.004	0.540
STLSort	0.000	0.004	0.523

Cuadro 2: Tiempos de ejecución empíricos (en segundos) para algoritmos de ordenamiento (Distribución Aleatoria).

3.2. Ordenamiento: Optimización de QuickSort (Antes vs Después)

•**Antes del ajuste:** La implementación original seleccionaba un pivote en el extremo del arreglo, lo que generaba un árbol de recursión completamente desbalanceado al enfrentar datos ascendentes o descendentes. Este peor caso obligaba al algoritmo a ejecutar un número de operaciones cercano a $O(n^2)$, volviéndolo tan ineficiente en la práctica como BubbleSort o SelectionSort y provocando tiempos de ejecución excesivos.

•**Después del ajuste:** Al modificar la selección del pivote hacia el centro del subarreglo, permite generar un árbol de recursión balanceado. Esta simple optimización ayuda a que se vuelva a un rendimiento

promedio del algoritmo a $\Theta(n \log n)$.

Tras la corrección, QuickSort logró ordenar los 10.000.000 de elementos en fracciones de segundo, posicionándose por debajo de las curvas de tiempo respecto a los otros algoritmos.

3.3. Ordenamiento: Optimización de PatienceSort (Antes vs Después)

Antes del ajuste: El diseño inicial dependía de búsquedas lineales y eliminaciones directas en el medio de arreglos dinámicos. Eliminar elementos ubicados entre los extremos de una estructura lineal es sumamente ineficiente, requiriendo un tiempo de $O(n)$ en el peor de los casos para desplazar la memoria restante. La repetición de estos desplazamientos degradaba todo el proceso a un tiempo cuadrático $O(n^2)$.

Después del ajuste: Se implementó una cola de prioridad basada en la estructura de un Min-Heap (priority queue). La teoría respalda que un Heap permite extraer el elemento de máxima prioridad (el mínimo) en un tiempo óptimo de $O(1)$ e insertar nuevos datos reorganizando el árbol en tiempo $O(\log n)$.

Reemplazar la búsqueda lineal por búsqueda binaria (lower bound) redujo el costo de inserción, estabilizando el tiempo global de la ejecución a un comportamiento predecible y eficiente de $O(n \log n)$.

Observacion: AL ejecutar el `sorting.cpp` el cual ejecutaba todos los algoritmos, hice un intento considerando las versiones no optimizadas de QuickSort y PatienceSort, lo deje andando por 12 horas y al despertarme aun seguía andando, por lo que decidí analizar sus versiones optimizadas.

3.4. Ordenamiento: STLSort y el poder de los algoritmos híbridos

Al observar la Tabla 2, destaca que STLSort (la implementación estándar del lenguaje) obtiene el mejor rendimiento general para $N = 10^7$ con 0.523 segundos, superando ligeramente al QuickSort optimizado (0.540 segundos). Esta ventaja radica en que las bibliotecas estándar modernas implementan algoritmos híbridos, comúnmente *IntroSort*. Este enfoque comienza ejecutando QuickSort para aprovechar su velocidad promedio, pero si detecta que la profundidad de recursión excede $2 \log n$, cambia dinámicamente a HeapSort para garantizar un peor caso de $O(n \log n)$, y utiliza InsertionSort para sub-arreglos muy pequeños donde las constantes operativas son más bajas. La estabilización geométrica en la Figura 2 confirma la superioridad técnica de no depender de una sola estrategia de ordenamiento.

3.5. Impacto de la Localidad de Caché (Cache Locality)

Un factor determinante que la notación teorica Big-O ignora es la interacción con la arquitectura de memoria del hardware. QuickSort y STLSort procesan los datos *in-place* mediante punteros que avanzan secuencialmente. Esto genera una alta *localidad espacial*, permitiendo que el procesador cargue bloques contiguos de datos en la memoria caché (L1/L2) minimizando los tiempos de búsqueda.

Por el contrario, algoritmos como PatienceSort, a pesar de su optimización con Min-Heap, fragmentan el acceso a la memoria. Como se evidencia en la Tabla 2, PatienceSort es el algoritmo más lento para $N = 10^7$ (1.159 s). La necesidad de saltar constantemente entre distintos nodos del Heap y las cimas de

múltiples pilas dinámicas (vectores) destruye la localidad de caché, forzando a la CPU a buscar datos en la memoria RAM principal repetidamente.

3.6. Consumo Espacial: Sobrecarga en la mezcla y estructuras

A pesar de que MergeSort, PatienceSort y QuickSort comparten una complejidad temporal eficiente, la Tabla 2 ubica a los dos primeros con tiempos absolutos superiores al segundo. Esta brecha ocurre por el manejo de la memoria extra. MergeSort exige la creación de arreglos auxiliares en cada nivel de recursión, sumando exactamente $2n$ (obtenido mediante el stack de recursión) asignaciones físicas para trasladar elementos, generando un costo estricto de memoria adicional de $O(n)$. PatienceSort sufre una penalización similar por instanciar múltiples arreglos dinámicos para sus colas. QuickSort opera aislando intercambios directamente sobre las direcciones originales, manteniendo una huella extra mínima de $O(\log n)$ (exclusiva de los llamados en la pila de ejecución recursiva), lo que alivia el trabajo del administrador de memoria del sistema y acelera el tiempo absoluto de ejecución.

3.7. Estabilidad vs Rendimiento

A pesar de la evidente superioridad en la experimentación (empírica) de QuickSort y STLSort en términos de tiempo de ejecución y consumo espacial (memoria), existe un factor teórico crítico que no se refleja en las gráficas de rendimiento absoluto: la *estabilidad* del algoritmo.

Un algoritmo de ordenamiento es estable si garantiza que los elementos con claves idénticas mantendrán su orden relativo original tras el procesamiento. MergeSort es un algoritmo inherentemente estable gracias a la forma secuencial en que mezcla los subarreglos. Por el contrario, el enfoque *in-place* de QuickSort, debido a sus intercambios dinámicos, es inestable. De igual manera, PatienceSort (al extraer cimas desde el Min-Heap) y la implementación estándar de STLSort (basada en IntroSort) carecen de estabilidad. De hecho, cuando el desarrollo en C++ requiere conservar el orden relativo, se debe sacrificar la velocidad pura de `std::sort` y utilizar explícitamente `std::stable_sort` (cuya implementación suele basarse precisamente en MergeSort).

Este atributo es fundamental. En contextos donde los registros poseen múltiples campos y requieren ordenamientos encadenados (por ejemplo, ordenar una base de datos de pacientes primero por fecha de ingreso y luego por prioridad médica), la estabilidad es un requerimiento necesario. En estos escenarios, el costo adicional de $O(n)$ en memoria y el ligero incremento en los tiempos de ejecución que presenta MergeSort dejan de ser una desventaja, convirtiéndose en el precio necesario a pagar para mantener la integridad relacional de los datos. Esto demuestra que la elección del algoritmo óptimo no depende únicamente de la velocidad pura, sino de las restricciones del dominio del problema.

4. Conclusiones

4.1. Respecto a algoritmos de Multiplicacion de Matrices

El análisis experimental demuestra que una complejidad asintótica teórica superior no garantiza velocidad inmediata en la práctica debido al impacto de las constantes ocultas. En la multiplicación de matrices, la sobrecarga recursiva de Strassen satura el procesador tempranamente, permitiendo que las operaciones directas del método Naive sean más veloces para la dimensión máxima exigida de $N = 2^8$. No obstante, la regla del límite asintótico establece que Strassen dominará inevitablemente a medida que el tamaño de entrada tienda al infinito.

4.2. Respecto a algoritmos de Ordenamiento

Por su parte, en los algoritmos de ordenamiento, el rendimiento depende críticamente de la gestión de memoria física y la mitigación de peores casos. QuickSort supera empíricamente a MergeSort porque opera in-place, evitando las costosas $2n$ asignaciones adicionales de memoria por nivel de recursión que MergeSort exige para mover datos. Además, reestructurar la lógica de los algoritmos previene su colapso computacional: centrar el pivote en QuickSort fuerza un árbol balanceado que evita su degradación a $O(n^2)$, mientras que la integración de un Min-Heap en PatienceSort garantiza la extracción del mínimo en un tiempo óptimo de $O(1)$. Finalmente, la inclusión del algoritmo SORT de la librería estándar de C++ sirvió como el punto de referencia definitivo dentro de las mediciones experimentales, demostrando empíricamente la brecha de rendimiento que existe entre una implementación estandarizada y los algoritmos construidos manualmente.

5. Implementación y Código Fuente

<https://github.com/ShirouXz/Tarea-1-INF-221.git>